Método de Newton Raphson

Javier Alexander López Contreras
Facultad de Ciencias Físico Matemáticas (FCFM), Universidad Autónoma de Coahuila, Campus Campo redondo

Recibido el

En este documento se analizará el código en Python de el método de Newton-Raphson; que es un método que utiliza un valor en x para encontrar la raíz de un polinomio.

1. INTRODUCCIÓN

El método de Newton-Raphson es de los mejores métodos para encontrar las raíces de un polinomio, ya que en general es muy eficiente y siempre converge para una función polinomial. A diferencia de los métodos cerrados (como bisección o regla falsa), este es un método abierto que requiere de un solo punto de partida.

2. ALGORITMO Y EXPLICACIÓN DEL MÉTODO DE .....

El método requiere que las funciones sean diferenciables, y por ende continuas, para poder aplicar este método.

El método consigue una aproximación trazando una recta tangente a la función en el punto inicial. Para encontrar la nueva aproximación, se realiza el siguiente algoritmo empleando la primera derivada de la función.

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \tag{1}$$

Al igual que en otros algoritmos numéricos, el código gestiona la cuantificación de errores para determinar la precisión de la aproximación actual.

$$E_{rel} = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \tag{2}$$

3. ANÁLISIS DEL CÓDIGO

El código se desarrolló en el lenguaje Python e implementa una interfaz gráfica de usuario utilizando la librria **tkinter** y **ttk**, lo que facilita la interacción, entrada de datos y visualización de los resultados.

3.1. Configuración de la interfaz y captura de la función

Para capturar la función introducida por el usuario, se utiliza un objeto **Entry**. Para interpretar la cadena de texto matemáticamente, se diseñó la función $f(x)$ que preprocesa la entrada reemplazando el símbolo de potencia clásico por el formato nativo de Python (**). Además, se evalúa mediante un entorno seguro contexto para mapear funciones trigonométricas y trascendentales a la librería **numpy**.

```python
    def f(x):
    formula = funcion.get().replace('^', '**')
    contexto = {
        'x': x, 'sin': np.sin, 'cos': np.cos,
        'log': np.log, 'sqrt': np.sqrt, 'pi': np.pi # ... entre otras
    }
    return eval(formula, contexto)

```

Listing 1: captura de la función y configuración

3.2. Visualización gráfica

Se implementó una rutina de graficación utilizando matplotlib.pyplot. El código incluye una validación inteligente dentro de la función grafica() que detecta si la función ingresada contiene un logaritmo (if 'log' in funcion.get():). Si es así, ajusta el dominio de la función a $x \in [0.1, 10]$ para evitar evaluar números negativos o el cero, previniendo errores de dominio matemático. Si no, grafica en un rango estándar de $[-10, 10]$.

```python
  def grafica():
    try:
        # Si es un logaritmo, ajustamos el rango para que no truene con negativos
        if 'log' in funcion.get():
            x = np.linspace(0.1, 10, 400)
        else:
            x = np.linspace(-10, 10, 400)

        y = f(x)
        plt.axhline(0, color='blue', lw=0.5)
        plt.axvline(0, color='blue', lw=0.5)
        plt.plot(x, y)
        plt.title(f'Gr fica de f(x) = {funcion.get()}')
        plt.grid()
        plt.show()
    except Exception as e:
        messagebox.showerror("Error de Gr fica", f"No se pudo graficar: {e}")
    }
    return eval(formula, contexto)

```

Listing 2: Rutina de graficación con Matplotlib y validación de dominio

3.3. Implementación del método de Newton-Raphson y derivada

A diferencia de otros scripts que requieren la derivada analítica programada manualmente, este código implementa una función genérica para calcular la derivada numérica utilizando el método de diferencias finitas centrales con un tamaño de paso muy pequeño ($h = 1 \times 10^{-6}$)

```
def derivada(x):
    h = 1e-6
    return (f(x+h)-f(x-h)) / (2*h)

```

Listing 3: Implementación de derivada por diferencias finitas

El algoritmo principal, encapsulado en def newton(), recupera los parámetros de tolerancia y máximo de iteraciones. El ciclo iterativo principal obedece la condición while error$_abs > tolanditeraciones < max$iter:. Dentro del ciclo se calcula la fórmula iterativa fundamental del método y sus respectivos errores:

```
x_nuevo = x_ant - (f(x_ant)/dfx)
error_abs = abs(x_nuevo - x_ant)
error_rel = abs(error_abs / x_nuevo) if x_nuevo !=
     0 else 0

```

Listing 4: Algoritmo principal de Newton-Raphson y cálculo de errores

4. RESULTADOS Y ANÁLISIS

En esta etapa se evaluó el desempeño del software frente a funciones con comportamientos específicos para poner a prueba la precisión del algoritmo

4.1. Caso $x^x - 100$ con $x = 0$

Para contrastar el manejo de errores, se realizó una prueba crítica utilizando la función $f(x) = x^x - 100$ con un valor inicial de $x = 0$, una tolerancia de 0.0001 y un límite de 100 iteraciones.Aunque el valor ingresado fue cero, el cálculo interno de la derivada numérica por diferencias finitas requiere evaluar $f(x - h)$. Como el tamaño de paso definido en el código es $h = 1 \times 10^{-6}$, el programa evalúa internamente $f(-1 \times 10^{-6})$. Matemáticamente, elevar un número negativo a una potencia fraccionaria produce un número complejo. El motor de Python gestionó esta operación de forma nativa en el plano complejo, provocando que el algoritmo divergiera. El ciclo iterativo terminó de forma controlada al llegar al límite de intentos, mostrando el mensaje: "Se alcanzó el máximo de iteraciones". La tabla de datos permaneció vacía, ya que la lógica del código exige convergencia real para mostrar los resultados.

Figura 1: Prueba con $x^x - 100$

4.2. Error por derivada nula (pendiente 0)

El método de Newton-Raphson falla geométricamente si la recta tangente a la curva es totalmente horizontal. Para poner a prueba la protección del código contra divisiones por cero, se evaluó la función $f(x) = x^2 - 4$ proponiendo un valor inicial de $x = 0$, con una tolerancia de 0.0001 y máximo de 100 iteraciones.Dado que el valor inicial coincide exactamente con el vértice de la parábola, la pendiente calculada es nula. El programa verificó exitosamente la condición de seguridad (if dfx == 0:), interrumpió el proceso antes de realizar el cálculo iterativo y notificó al usuario imprimiendo el texto en rojo: "La derivada es 0 en x = 0.0".

Figura 2: Prueba con $x^2 - 4$

4.3. Convergencia exitosa con $x^x - 100$

Finalmente, se procedió a buscar la raíz real de la función $f(x) = x^x - 100$, introduciendo una semilla matemáticamente apta de $x = 3$, conservando la tolerancia de 0.0001 y el límite de 100 iteraciones.El método de Newton-Raphson demostró su eficiencia y convergencia cuadrática al encontrar la raíz aproximada en $x = 3.597285$ en tan solo 7 iteraciones. La tabla visual (Treeview) se pobló con los datos iterativos, mostrando cómo el error absoluto disminuyó drásticamente desde $1.29\text{e} + 00$ en la primera iteración hasta $1.28\text{e} - 07$ en la última, y el sistema validó el proceso con el mensaje de éxito en verde.

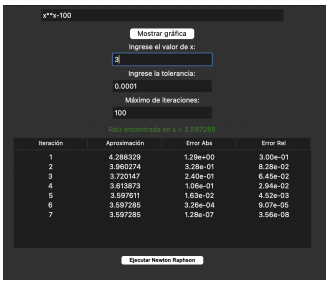

Figura 3: Prueba con $x^x - 100$ con $x = 3$

5. CONCLUSIONES

El método de Newton-Raphson es una herramienta superior por su rapidez matemática, pero esto depende de la correcta selección del valor inicial y de que la función no presente indeterminaciones en su derivada. El análisis del código demuestra que no basta con implementar las fórmulas correctas; el manejo de excepciones debe ser estricto. Reemplazar la instrucción **break** por una interrupción total del sistema evitaría la generación de reportes tabulares con datos complejos o imaginarios cuando el método fracasa en la primera iteración.

6. Bibliografía

[1] Chapra, S. C., y Canale, R. P. (2015). Métodos numéricos para ingenieros (7a ed.). México: McGraw-Hill. [2] Nieves, A., y Domínguez, F. C. (2014). Métodos numéricos aplicados a la ingeniería (4a ed.). México: Grupo Editorial Patria.